

IIT HYDERABAD

CS5903 – Distributed AI Training

AdaptEngine

Adaptive Learning, Diagnostic Assessment and
Personalized Path-Planning Engine

Week 2 Progress Report

Student: Vuppula Hemanth Reddy

Roll No.: AI23BTECH11033

Group: 15

Repository: <https://github.com/vhemanth06/AdaptEngine>

Contents

1	Introduction	3
2	Response to Week 1 Feedback and updates	3
2.1	Select One Primary Mastery Model	3
2.2	Remove Unnecessary Custom Search	4
2.3	Refinement of Mastery and Uncertainty	4
2.4	Apply Agentic AI if neccesary	4
2.5	Add a Review Gate for Curriculum Content	4
2.6	Revised Test and Traceability Plan	5
3	Architecture Design	7
3.1	Design Principles	7
3.2	Layered System Architecture	7
3.2.1	Core Layer	8
3.2.2	Domain Layer	8
3.2.3	Decision Layer	8
3.2.4	Orchestration Layer	8
3.2.5	Interfaces Layer	8
3.2.6	Optional Agentic Layer	9
3.3	System Context	9
3.4	Curriculum as a Directed Acyclic Graph	9
4	Mastery Model	10
4.1	Initial Mastery	10
4.2	Difficulty-Dependent Guess and Slip	10
4.3	Bayesian Response Update	11
4.4	Example	11
4.5	Mastery and Uncertainty	11
5	Week 2 Implementation	12
5.1	Implementation Scope	12
5.2	Tests Passed in Week 2	12
5.3	Schema and Validation	12
5.4	Typed Errors	13
5.5	Curriculum Review Gate	13
5.6	Learner State	13
5.7	Event Model	14
5.8	Public Engine API	14
6	Configuration and Reproducibility	14
7	Testing Strategy	15
7.1	Testing Philosophy	15
7.2	Ground-Truth Difficulty Values	16
7.3	Ground-Truth Mastery Updates	16
7.4	Invalid Inputs	16
8	Milestone Mapping	17

9 Week 2 Outcomes	17
9.1 Engineering Outcomes	17
9.2 Why This Foundation Matters	18
10 Conclusion	18
11 Reference Mastery Calculations	18
11.1 Guess and Slip	18
11.2 Correct Response	18
11.3 Incorrect Response	19
12 Current Status Summary	19

1 Introduction

Project Motivation

Learners differ in prior knowledge, available study time, learning speed, and deadlines. A fixed course sequence therefore cannot efficiently serve all learners. AdaptEngine is designed to solve this problem by maintaining a representation of learner knowledge and using it to determine what should be learned next.

The complete target system spans milestones M1–M8:

- **M1:** curriculum representation and learner state;
- **M2:** adaptive diagnostic assessment;
- **M3:** prerequisite-valid and time-budgeted planning;
- **M4:** online mastery update and difficulty adaptation;
- **M5:** spaced review and remediation;
- **M6:** event-driven replanning;
- **M7:** deterministic decision traces and explanations;
- **M8:** evaluation against fixed and static baselines.

The objective of Week 2 was to implement the core components of **M1 (Curriculum and Learner State)** and partially implement **M4 (Online Mastery Update and Difficulty Adaptation)**, while designing the overall architecture for milestones M1–M8. The focus was therefore on establishing a working foundation for curriculum representation, prerequisite relationships, learner state and response-driven mastery updates, while defining the interfaces and layered architecture required for the subsequent milestones.

Week 2 Objective

Deliver a small working foundation that can load and validate a reviewed curriculum, construct its prerequisite graph, create learner state, accept typed events, update mastery, and provide deterministic results through a single engine interface.

2 Response to Week 1 Feedback and updates

The Week 1 review concluded that the conceptual proposal was strong, but implementation had not yet demonstrated correctness. The review also highlighted that implementing custom A* search, diagnostic selection, mastery modelling, and simulator calibration simultaneously would create unnecessary delivery risk.

The major Week 2 changes are summarized below.

2.1 Select One Primary Mastery Model

The Week 1 review highlighted a conceptual distinction between mastery probability and uncertainty. The revised design therefore treats the BKT-style mastery probability as the primary response-driven state representation rather than attempting to combine multiple mastery models.

The Week 2 implementation uses the project-defined BKT-style update, while alternative models remain future comparison points.

2.2 Remove Unnecessary Custom Search

The original design considered custom A* search for scheduling. This was identified as avoidable implementation risk.

The revised architecture instead targets a deterministic scheduler with explicit feasibility constraints. A tiny exact oracle is planned for small instances as an independent correctness reference.

Thus:

hard constraints $\rightarrow$ eligible activities $\rightarrow$ deterministic ranking $\rightarrow$ schedule.

The learned or heuristic ranking mechanism is never responsible for checking whether a prerequisite or time-budget rule has been violated.

2.3 Refinement of Mastery and Uncertainty

The Week 1 feedback highlighted the need to clearly define **mastery** and **uncertainty**. We therefore revised the mastery update to a response-driven BKT-style formulation in which the learner's mastery probability is updated using the current mastery, the activity difficulty, and difficulty-dependent guess and slip probabilities.

For an activity with difficulty d , we use

$$g(d) = 0.30(1 - d), \quad s(d) = 0.05 + 0.25d,$$

and update the mastery probability differently for correct and incorrect responses. A learning transition is then applied after the response update.

We also separated **uncertainty** from mastery instead of treating the mastery probability itself as a measure of uncertainty. Uncertainty is represented using the entropy of the mastery probability:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p).$$

Thus, mastery answers “How likely is the learner to have mastered the concept?”, while entropy answers “How uncertain are we about that mastery estimate?”. This distinction provides a clearer foundation for the adaptive diagnostic component planned for M2.

2.4 Apply Agentic AI if necessary

Architecture limits the optional agentic layer to four bounded roles: Plan Proposer, Ranking Policy, Orchestrator, and Authoring Aid. The deterministic engine remains the authority over validity, mastery, feasibility, and state transitions.

Agents are therefore optional and placed above the deterministic engine rather than inside its critical decision path.

2.5 Add a Review Gate for Curriculum Content

A new review-manifest mechanism was introduced into the design. Live curriculum files must have a corresponding entry in `data/review_manifest.json`. The manifest records review metadata including a SHA-256 hash.

The resulting pipeline is:

draft content $\rightarrow$ human review $\rightarrow$ hash recorded $\rightarrow$ live curriculum.

A modification to a reviewed file changes its hash and causes the review gate to reject the content until it is reviewed again.

2.6 Revised Test and Traceability Plan

Instead of using a single sequential test number, each core test case is now identified by its milestone and sequence number. For example, TC1.1 denotes the first test case for M1, while TC4.2 denotes the second test case for M4. This makes the relationship between implementation, milestone, and verification immediately visible.

TC-X3.1, where X3 refers to the A3 Plan Proposer extension.

Table 1: Revised test register and milestone traceability

ID	Scenario / Input	Expected Result	Week
M1 – Curriculum and Learner State			
TC1.1	Load the prerequisite graph and inject a cycle.	A valid graph is confirmed acyclic; a cycle is rejected at load time with a typed error identifying the invalid concepts.	W2
TC1.2	Load valid and invalid curriculum and learner-state schema fixtures.	Valid fixtures load with the correct types; invalid types, ranges, unknown references, and missing required fields raise the appropriate typed validation errors.	W2
M2 – Adaptive Diagnostic			
TC2.1	Run the adaptive diagnostic on the four-node and larger curriculum fixtures.	The diagnostic stops when the uncertainty threshold or item cap is reached and never exceeds $N_{\max}$ selected items.	W3
TC2.2	Run diagnosis for concepts with no previous observations.	Previous learner state is preserved, the concept is marked UNASSESSED, and it is treated as unmastered for diagnostic purposes.	W3
TC2.3	Two learners with different prior mastery are given the same goal and budget.	They receive different, individually valid plans appropriate to their learner states.	W3–W4
M3 – Personalized Planning and Feasibility			
TC3.1	Schedule a plan on a graph with an unmet mandatory prerequisite.	No lesson for the locked concept is scheduled before its prerequisite is satisfied.	W4
TC3.2	Generate a plan under a fixed daily time budget.	No day’s total scheduled duration exceeds the configured daily budget.	W4
TC3.3	Use the same learner and goal with a 5-minute budget versus a 30-minute budget.	The resulting plans are materially different in pacing and/or number of days.	W4
TC3.4	The learner’s goal cannot fit before the deadline under the available daily budget.	The engine returns an INFEASIBLE result with the shortfall and corrective options rather than producing an invalid plan.	W4
TC3.5	Run the 8 + 8 + 4 packing fixture with a 10-minute daily budget.	The feasibility bounds pass while the exact oracle reports INFEASIBLE, since the workload requires three days.	W4
TC3.6	Force the feasibility search to exceed its configured time limit.	The oracle returns UNKNOWN rather than incorrectly reporting INFEASIBLE.	W4

ID	Scenario / Input	Expected Result	Week
M4 – Online Mastery Update and Difficulty			
TC4.1	A learner gives an incorrect response on a low-mastery concept.	The mastery estimate updates downward according to the response model; for $p = 0.5$, $d = 0.5$, the reference Bayesian update gives $p \approx 0.1707$.	W2–W3
TC4.2	Select the next exercise for a learner with a given mastery estimate.	The exercise whose difficulty is nearest to the target mastery is selected, with ties resolved by the specified difficulty-policy rule.	W3
M5 – Review and Remediation			
TC5.1	A concept is already mastered above the mastery threshold θ .	The concept is omitted from new-learning scheduling, with the decision trace citing the mastery evidence.	W5
TC5.2	A failed exercise leaves the learner with $p < 0.40$.	A remediation lesson is inserted; after the specified remediation cycles, the prerequisite review state and remediation flag are updated correctly.	W5
TC5.3	A concept has high review urgency while spare daily capacity exists.	The review activity receives priority over lower-urgency new content.	W5
M6 – Replanning			
TC6.1	The learner misses a scheduled day.	Replanning produces a new valid plan without losing or duplicating completed work.	W5
TC6.2	Trigger a failed-assessment event.	The learner state is updated and a valid revised plan is generated with the required remediation response.	W5
TC6.3	Change the learner’s deadline after a plan has already been generated.	The engine produces a valid revised plan or proves that the updated goal is INFEASIBLE .	W5
TC6.4	Change the learner’s available daily time budget.	The engine produces a valid revised plan that satisfies the new daily budget.	W5
M7 – Decision Traces and Explanations			
TC7.1	A generated plan contains both selected and omitted activities.	Every decision is traceable to learner state, constraints, and score components.	W5–W6
M8 – Evaluation			
TC8.1	Run the same evaluation harness with identical seeds and configuration twice.	The raw results are identical and the frozen baselines remain unchanged.	W6
TC8.2	Compare frontier-based replanning against full recomputation on the same workload events.	Both approaches produce identical decisions on the test workload and their replanning latency is recorded for comparison.	W6

3 Architecture Design

3.1 Design Principles

Architecture is based on eight primary quality attributes:

Table 2: Architecture quality attributes

#	Attribute	Meaning
Q1	Constraint safety	Prerequisites and daily budgets are enforced as hard constraints.
Q2	Determinism	The same inputs and configuration produce the same decisions and traces.
Q3	Traceability	Decisions contain machine-readable reasons.
Q4	Testability	Modules are independently testable and deterministic where possible.
Q5	Replaceability	Mastery, review, ranking and planning components can be replaced through narrow interfaces.
Q6	Simplicity	The design remains practical for a single developer.
Q7	Measurable performance	Important operations can be benchmarked on fixed workloads.
Q8	Bounded autonomy	Agents may propose or rank options but cannot bypass engine constraints.

3.2 Layered System Architecture

AdaptEngine follows a layered architecture in which each layer has a specific responsibility. The lower layers provide stable deterministic building blocks, while higher layers combine these components to implement learning and planning behaviour. The optional agentic layer is kept outside the deterministic core.

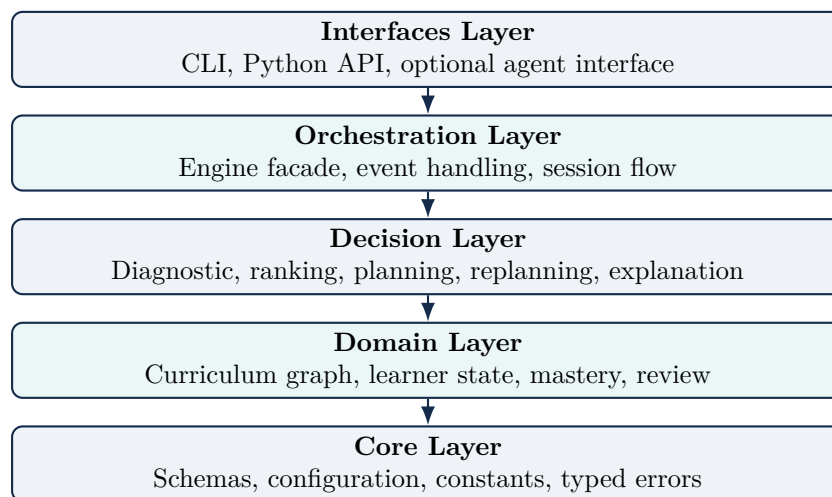


Figure 1: Layered architecture of AdaptEngine

3.2.1 Core Layer

The Core layer contains the basic structures and rules shared by the entire system. It defines typed schemas, configuration values, constants, and the project's error hierarchy. This layer does not contain learning or planning logic.

Its main purpose is to provide stable and reusable primitives for the layers above it. Numerical parameters such as the initial mastery probability, mastery threshold, learning-transition values, and difficulty bounds are centralized in the configuration rather than being scattered throughout the implementation.

3.2.2 Domain Layer

The Domain layer represents the actual state of the learning problem. It contains the curriculum, prerequisite relationships, learner state, mastery information, and review-related state.

The curriculum is represented as a directed acyclic graph (DAG), where nodes represent concepts and edges represent prerequisite relationships. The learner model maintains the current mastery of each concept and stores the information required for future updates and review.

For Week 2, this layer is the main implementation focus through the curriculum loader, prerequisite graph, learner state, and mastery update components.

3.2.3 Decision Layer

The Decision layer contains the algorithms that determine what the engine should do. In the complete architecture, this layer is responsible for adaptive diagnostic selection, activity ranking, planning, replanning, review decisions, and deterministic explanations.

The important design principle is that decision components operate on the validated domain state. Hard constraints such as prerequisite validity and time budgets are enforced by the engine and are not left to a learned or heuristic ranking component.

3.2.4 Orchestration Layer

The Orchestration layer connects the domain state and decision components and controls how an operation is executed.

The main entry point is the **Engine** facade. Events are received as typed objects and passed through the engine, which applies the appropriate state transition and records the execution information.

The public operation

```
Engine.record(state, log, event)
```

provides a single controlled interface for state-changing operations. This keeps the workflow consistent and makes later features such as replanning and simulation easier to integrate.

3.2.5 Interfaces Layer

The Interfaces layer provides the external entry points to AdaptEngine. The Week 2 implementation contains a thin command-line interface and a Python API.

The interface layer should remain lightweight: it translates external inputs into typed engine operations and presents the resulting state or errors. Learning and planning logic should not be implemented directly inside the CLI or API.

This separation allows the same deterministic engine to be used from a CLI, Python program, simulator, or future application without duplicating business logic.

3.2.6 Optional Agentic Layer

The architecture also defines an optional agentic layer above the deterministic core. It is limited to four bounded agents, each with a specific role:

- **A1–Plan Proposer:** Attempts to generate a customized day-by-day learning plan based on the learner’s state, goals, prerequisites, available activities, budget and deadline. The proposed plan is always validated by the engine before it can be accepted.
- **A2–Ranking Policy:** Reorders activities that have already passed the engine’s hard-constraint filter. It can rank valid activities according to engagement or efficiency, but cannot introduce new activities or violate prerequisite constraints.
- **A3–Orchestrator:** Chooses the overall learning strategy, such as continuing the current plan, applying remediation, performing review, or re-diagnosing the learner. It selects only from a closed set of options generated and validated by the engine.
- **A0–Authoring Aid:** Acts as an offline development assistant for drafting curriculum content such as concept relationships, lesson text and remediation hints. Its output is treated as draft content and must go through human review before entering the live curriculum.

The central principle is that agents propose, reorder, or choose, while the engine validates and decides. Agents are therefore not authoritative over curriculum validity, learner mastery, or hard scheduling constraints. They are also disabled by default, allowing the system to operate without network access and providing a deterministic baseline for testing and evaluation.

3.3 System Context

At the system level, learners interact with the engine through a thin CLI or Python API. Optional agents may request engine decisions, but the deterministic core remains authoritative.

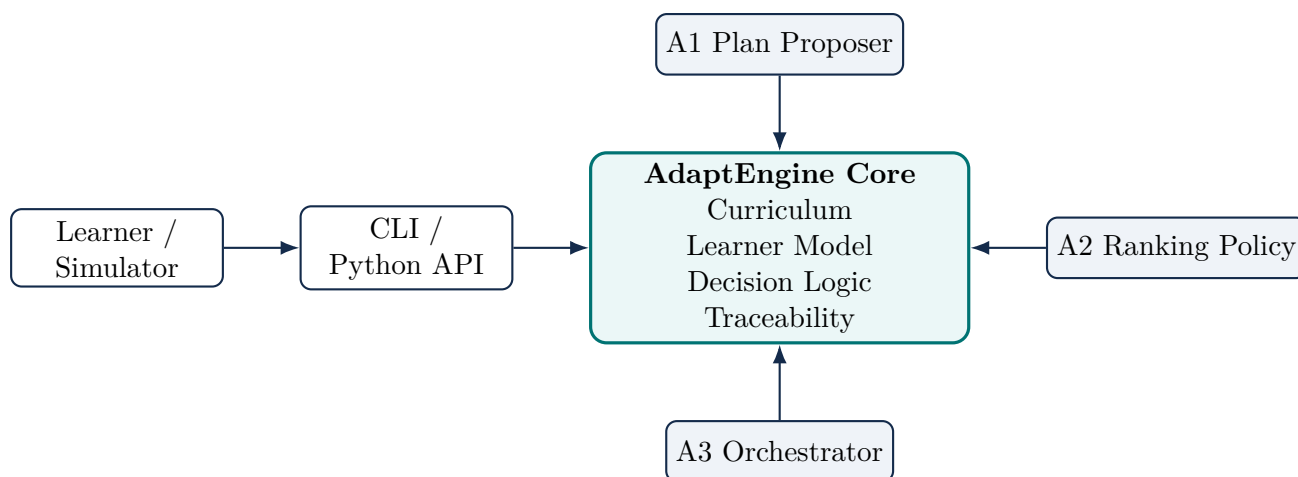


Figure 2: System context and bounded optional agents

3.4 Curriculum as a Directed Acyclic Graph

Each concept has an identifier, description, and prerequisite set. Prerequisites form a directed acyclic graph (DAG).

For the initial four-node fixture:

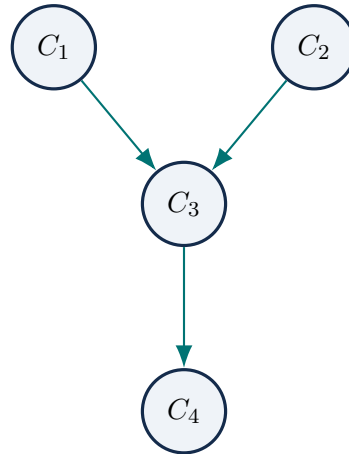


Figure 3: Four-concept prerequisite join used as the Week 2 fixture

The graph layer is responsible for operations such as:

- checking that concept identifiers are unique;
- checking that prerequisite references exist;
- checking graph acyclicity;
- finding ancestors and prerequisite chains;
- producing stable topological ordering.

Display-oriented fields such as concept descriptions must not alter the graph structure.

4 Mastery Model

4.1 Initial Mastery

The initial mastery probability is defined as

$$p_0 = 0.20.$$

This value is a project configuration default rather than a measured parameter.

4.2 Difficulty-Dependent Guess and Slip

For activity difficulty d , with

$$d \in [0.05, 0.95],$$

the proposed guess and slip functions are

$$g(d) = 0.30(1 - d)$$

and

$$s(d) = 0.05 + 0.25d.$$

Thus easier exercises have a higher probability of being answered correctly by a learner who has not mastered the concept, while incorrect responses become increasingly plausible as difficulty increases.

4.3 Bayesian Response Update

Let p be the current probability that the learner has mastered the concept.

For a correct response:

$$p' = \frac{p(1-s)}{p(1-s) + (1-p)g}.$$

For an incorrect response:

$$p' = \frac{ps}{ps + (1-p)(1-g)}.$$

A small learning transition is then applied:

$$p'' = p' + (1-p')T,$$

where T depends on event type.

The Week 2 defaults are:

$$T_{\text{diagnostic}} = 0,$$

$$T_{\text{exercise}} = 0.05,$$

$$T_{\text{lesson}} = 0.20.$$

4.4 Example

For

$$p = 0.5, \quad d = 0.5,$$

we obtain

$$g(0.5) = 0.15, \quad s(0.5) = 0.175.$$

For a correct response without the learning transition,

$$p' = \frac{0.5(1-0.175)}{0.5(1-0.175) + 0.5(0.15)} \approx 0.8462.$$

For an incorrect response,

$$p' = \frac{0.5(0.175)}{0.5(0.175) + 0.5(1-0.15)} \approx 0.1707.$$

These values form part of the independent test constants used to verify the implementation.

4.5 Mastery and Uncertainty

Mastery probability is not treated as a complete representation of uncertainty. The architecture therefore also defines entropy as a simple uncertainty measure:

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

Important reference values include

$$H(0.5) = 1.0,$$

$$H(0.2) \approx 0.7219,$$

$$H(0.9) \approx 0.4690.$$

This distinction becomes important later when diagnostic assessment needs to select activities that reduce uncertainty rather than merely maximize immediate mastery.

5 Week 2 Implementation

5.1 Implementation Scope

Week 2 concentrates on the following components:

Table 3: Week 2 implementation scope

Component	Week 2 scope
<code>core/</code>	Schemas, constants, configuration and typed errors
<code>curriculum/</code>	Curriculum loader, validation, review gate and graph operations
<code>learner/</code>	Learner state and mastery update logic
<code>session.py</code>	Event application and persistence helpers
<code>api.py</code>	Minimal <code>Engine</code> facade and public <code>record</code> API
<code>cli.py</code>	Thin command-line interface
Tests	Schema, loader, graph, mastery, state, session, API and architecture tests
CI	Automated linting and test execution
Documentation	Architecture, schema, model specification and reuse inventory

5.2 Tests Passed in Week 2

The following test cases from the revised traceability plan were completed and passed during Week 2:

- **TC1.1** – Prerequisite graph loading and cycle rejection.
- **TC1.2** – Curriculum and learner-state schema validation.
- **TC4.1** – Response-driven mastery update and entropy calculation.

5.3 Schema and Validation

The core schema defines the typed domain objects used by the system. Important fields include:

- `Concept.id`;
- `Concept.description`;
- `Activity.text`;
- `LearnerConcept.mastery`;
- `LearnerConcept.half_life_days`;
- activity difficulty;

- activity duration;
- prerequisite identifiers;
- typed event information.

Validation rejects malformed inputs before they enter the decision system.

5.4 Typed Errors

The architecture defines an explicit error hierarchy rather than allowing low-level exceptions to leak through the public API.

Examples include:

```
1 class AdaptEngineError(Exception):
2     pass
3
4 class ValidationError(AdaptEngineError):
5     pass
6
7 class DuplicateIdError(ValidationError):
8     pass
9
10 class UnknownConceptError(ValidationError):
11     pass
12
13 class DifficultyError(ValidationError):
14     pass
15
16 class UnreviewedContentError(ValidationError):
17     pass
18
19 class CycleError(AdaptEngineError):
20     pass
```

This enables the CLI and API to provide clear failure messages while keeping domain validation explicit.

5.5 Curriculum Review Gate

The loader is responsible for validating live curriculum files. A curriculum file is accepted only if its corresponding entry in the review manifest exists and the recorded SHA-256 hash matches the current file.

Conceptually:

`load(file) → validate(schema) → check_manifest(file) → construct_graph.`

The gate prevents an accidentally modified or unreviewed file from being treated as trusted curriculum content.

The manifest is deliberately separate from `docs/data_manifest.md`. The former controls content review; the latter documents data sources.

5.6 Learner State

Learner state stores the current state of each learner–concept pair. A simplified representation is:

```

1 LearnerConcept(
2     concept_id=...,
3     mastery=...,
4     half_life_days=...,
5     last_seen=...
6 )

```

The `half_life_days` field is included for compatibility with the future review model but does not drive Week 2 logic.

5.7 Event Model

All state transitions are triggered through typed events.

Representative events include:

- `COMPLETED`;
- `MISSED`;
- `FAILED_ASSESSMENT`;
- `DEADLINE_CHANGED`;
- `BUDGET_CHANGED`.

In Week 2, the main implemented pathway is response-driven mastery updates through the engine recording interface. Events that require future replanning are represented so that later milestones can use the same event abstraction.

5.8 Public Engine API

The architecture makes

$$\text{Engine.record}(\text{state}, \text{log}, \text{event})$$

the principal public mutation interface.

The method returns updated learner state and execution log:

$$(\text{state}, \text{log}) \leftarrow \text{Engine.record}(\text{state}, \text{log}, \text{event}).$$

The design avoids exposing many unrelated mutation methods such as public `apply_response` and `apply_lesson`. Those become internal helpers behind a single event-processing boundary.

6 Configuration and Reproducibility

All numerical defaults are centralized.

Table 4: Week 2 model constants

Parameter	Symbol	Default
Initial mastery	p_0	0.20
Mastery threshold	θ	0.85
Diagnostic transition	T_{diag}	0.00
Exercise transition	T_{exercise}	0.05
Lesson transition	T_{lesson}	0.20
Guess coefficient		0.30
Slip base		0.05
Slip coefficient		0.25
Minimum difficulty	d_{min}	0.05
Maximum difficulty	d_{max}	0.95

The configuration object is immutable and its serialized representation is hashed using SHA-256:

```
config_hash = SHA256(JSON(asdict(config), sort_keys=True)).
```

The purpose of this design is reproducibility. A change to a numerical configuration should be visible in experiment records rather than being hidden inside implementation code.

7 Testing Strategy

7.1 Testing Philosophy

The major test categories are:

- schema and type validation;
- duplicate and unknown identifier detection;
- difficulty and duration validation;
- cycle detection;
- review-manifest validation;
- graph ordering and ancestor queries;
- mastery update values;
- entropy calculations;
- learner state transitions;
- `Engine.record`;
- CLI behaviour;
- architecture boundary rules.

7.2 Ground-Truth Difficulty Values

The following values serve as implementation-independent references.

Table 5: Expected guess and slip values

Difficulty d	$g(d)$	$s(d)$
0.05	0.285	0.0625
0.20	0.240	0.1000
0.40	0.180	0.1500
0.50	0.150	0.1750
0.60	0.120	0.2000
0.80	0.060	0.2500
0.95	0.015	0.2875

7.3 Ground-Truth Mastery Updates

Table 6: Reference mastery update values

p	d	Response	T	Expected p
0.50	0.50	Correct	0.00	0.8462
0.50	0.50	Incorrect	0.00	0.1707
0.50	0.50	Correct	0.05	0.8538
0.50	0.50	Incorrect	0.05	0.2122
0.90	0.50	Incorrect	0.00	0.6495

7.4 Invalid Inputs

The implementation includes invalid fixtures covering cases such as:

- duplicate concepts;
- duplicate activities;
- unknown prerequisites;
- self-loops;
- cyclic prerequisites;
- unknown activity targets;
- missing difficulty;
- out-of-range difficulty;
- invalid activity duration;
- malformed JSON;
- invalid schema versions.

8 Milestone Mapping

Week 2 focused on designing the overall M1–M8 architecture, implementing **M1**, and partially implementing **M4**. The remaining milestones were designed as part of the architecture and will be implemented incrementally in subsequent weeks.

Table 7: Relationship between milestones and Week 2 work

Milestone	Objective		Status
M1	Curriculum and learner state	Schema, validation, curriculum loader, review gate, prerequisite graph, learner state, and execution log.	Implemented
M2	Adaptive diagnosis	Mastery foundation and typed event interfaces provide the basis for future diagnostic integration.	Deferred
M3	Personalized planning	The prerequisite graph and validated activity metadata provide the foundation for future planning.	Deferred
M4	Online mastery and difficulty	A real response-driven mastery update is implemented. The difficulty-adaptation policy is planned for the next stage.	Partially implemented
M5	Review and remediation	Learner state contains the information required for future review and remediation logic.	Deferred
M6	Replanning	Typed events provide the foundation for handling future replanning triggers.	Deferred
M7	Explanations	The execution log provides a foundation for later deterministic decision traces.	Deferred
M8	Evaluation	The deterministic interfaces and smoke-test setup provide the foundation for future baseline and simulator experiments.	Deferred

9 Week 2 Outcomes

The main outcome of Week 2 is a transition from an architecture-only proposal toward a concrete engineering foundation.

9.1 Engineering Outcomes

The Week 2 work establishes:

1. a structured package hierarchy;
2. explicit core schemas and typed errors;
3. validated curriculum loading;
4. a content-review gate based on file hashes;
5. prerequisite graph construction and validation;
6. an explicit learner-state representation;
7. a real mastery-update function;
8. a typed event model;
9. the `Engine.record` public API;

10. deterministic smoke-test infrastructure;
11. architecture-level tests;
12. a CI-ready repository structure.

9.2 Why This Foundation Matters

Later components depend on these contracts.

For example, the future planner assumes that:

$$\text{curriculum} \rightarrow \text{validated DAG}$$

and

$$\text{learner responses} \rightarrow \text{typed event} \rightarrow \text{updated learner state}.$$

Once these interfaces are stable, M2–M8 can be added without changing the basic representation of curriculum and learner state.

10 Conclusion

Week 2 converts the most important parts of the AdaptEngine proposal into an explicit implementation foundation.

The most significant change from Week 1 is not the number of features implemented, but the reduction of uncertainty around the core interfaces and assumptions. The project now has a clearer separation between curriculum data, learner state, mastery updates, event processing, and future planning logic.

The architecture also establishes a strong deterministic centre: prerequisites, validity, learner-state transitions, and configuration remain under engine control, while optional learned or agentic components can later operate around this core.

The four-concept fixture, review gate, graph layer, learner state, mastery update, typed events, `Engine.record` interface, tests, and CI infrastructure together provide the base required for the next milestones. The remaining work is intentionally staged so that diagnosis, planning, review, replanning, explanation, and evaluation can be added and measured incrementally rather than being implemented as one large unverified system.

11 Reference Mastery Calculations

11.1 Guess and Slip

For $d = 0.5$:

$$g(0.5) = 0.30(1 - 0.5) = 0.15,$$

$$s(0.5) = 0.05 + 0.25(0.5) = 0.175.$$

11.2 Correct Response

With $p = 0.5$:

$$p' = \frac{0.5(1 - 0.175)}{0.5(1 - 0.175) + 0.5(0.15)}$$

$$= \frac{0.4125}{0.4875} \approx 0.8462.$$

With $T = 0.05$:

$$p'' = 0.8462 + (1 - 0.8462)(0.05) \approx 0.8538.$$

11.3 Incorrect Response

$$\begin{aligned} p' &= \frac{0.5(0.175)}{0.5(0.175) + 0.5(1 - 0.15)} \\ &= \frac{0.0875}{0.5125} \approx 0.1707. \end{aligned}$$

With $T = 0.05$:

$$p'' = 0.1707 + (1 - 0.1707)(0.05) \approx 0.2122.$$

12 Current Status Summary

Table 8: End-of-Week-2 status

Area	Status
Architecture	Defined in Architecture; deterministic core with optional bounded agents.
Curriculum schema	Week 2 foundation.
Curriculum validation	Week 2 foundation.
Review-manifest gate	Week 2 foundation.
Prerequisite graph	Week 2 foundation.
Learner state	Week 2 foundation.
BKT-style mastery update	Real update implemented as the M1/M4 foundation.
Diagnostic	Future milestone (M2).
Planner	Future milestone (M3).
Difficulty policy	Future milestone (M4).
Review/remediation	Future milestone (M5).
Replanning	Future milestone (M6).
Decision explanations	Future milestone (M7).
Simulator/evaluation	Future milestone (M8).
Agent layer	Future milestone.